

Threading

¿Qué es threading y qué problema resuelve?

El threading o mejor conocido como manejo de hilos, es el mecanismo que le permite a nuestro programa que ejecute múltiples flujos de control al mismo tiempo o de manera simultanea en un mismo proceso.

Lo que nos ayuda a resolver es el problema de la subutilización de recursos y lo bloqueos asíncronos, que en otras palabras se refieren a cuando no utilizamos toda la capacidad de nuestra máquina, pagando por la potencia que no usamos y a frenar por completo una fila de trabajo por la culpa de una sola tarea que se gestionó mal en su codificación.

Podemos resolver estos problemas, en el ejemplo de un modelo secuencial (que es un solo hilo), si el sistema quiere hacer una consulta lenta a una base de datos o descargar un archivo, toda nuestra aplicación va a congelarse en lo que espera una respuesta. En cambio, si le introducimos concurrencia, nuestro sistema puede delegar la tarea pesada a un hilo secundario y así mantenemos nuestro hilo principal libre para que este pueda seguir interactuando con ya sea usuarios o otras peticiones.

Evolución del manejo de hilos

Java 1.0 – Gestión manual (Thread y Runnable): Se creaban hilos extendiendo la clase Thread o implementando Runnable. Cada uno de estos hilos de Java se mapeaba 1:1 a un hilo del OS. El problema que tenia esto es que crear y destruir estos hilos era muy costoso en memoria y en ciclos de CPU.

Java 5 – La revolución de ExecutorService: Aquí se introdujo la API de threading (java.util.concurrent). Ya no se creaba un hilo por tarea ahora se crearon los Thread Pools. Nuestro ExecutorService puede mantener un flujo de hilos vivos; cuando llega una tarea, un hilo la ejecuta y cuando termina, vuelve al pool para ser reutilizado. Esto evita que el sistema colapse por la saturación de memoria.

Java 8 – CompletableFuture: Este modelo de threading ya era más funcional y asíncrono. En esta edición de java ya podemos encadenar tareas en múltiples hilos sin bloquear el sistema mientras esperamos los resultados, todo esto por la creación de pipelines de procesamiento de datos.

Java 21 – Virtual Threads (Project Loom): Este es el cambio mas reciente, en gran contraste con la problemática que teníamos en Java 1.0, ahora los hilos consumen 2MB de RAM cada uno, lo que limita a los servidores a unos pocos miles. Los hilos virtuales son gestionados por la Máquina Virtual de Java (JVM), y ya no por el OS. Ya podemos mantener millones ejecutándose al mismo tiempo lo que es ideal para aplicaciones web masivas.

¿Qué pasa si se implementa mal?

La concurrencia nos da problemas críticos si los hilos no se sincronizan correctamente al acceder a la misma información.

Condición de Carrera (Race condition): Esto pasa cuando dos hilos leen y modifican la misma variable al mismo tiempo.

Abrazo Mortal (Deadlock): Aquí es cuando nuestro hilo A bloquea al recurso 1 y necesita el recurso 2. El hilo B bloquea al recurso 1. Ninguno va a avanzar, y la aplicación se congela.

Mecanismos de Control

Synchronized: Esta es una palabra reservada que es segura pero lenta. Crea un cerrojo (lock) sobre un método o bloque de código. Si el hilo A entra, el hilo B tiene que esperarse hasta el el hilo A acabe.

Operaciones Atómicas (AtomicInteger, AtomicReference): A comparación con las synchronized estas no bloquean nuestros hilos y son más rápidas. Utilizan instrucciones que están a nivel hardware (Compare-And-Swap) para actualizar valores en un paso, sin corromper los datos.

Colecciones Concurrentes: Estas son las estructuras ConcurrentHashMap o CopyOnWriteArrayList, fueron diseñadas para que múltiples hilos las lean y escriban sin corromper la colección de datos y sin bloquearla por completo.

Serialización

¿Qué es y qué problema resuelve la serialización?

La serialización es tomar un objeto vivo en la memoria RAM (con todos sus atributos y estado actual) y lo convertimos en un archivo (paquete de bytes o datos). Lo congelamos.

La deserialización es lo contrario, tomamos ese archivo y lo convertimos en el objeto exacto dentro del programa, lo descongelamos.

El problema que resuelve es que los objetos de Java son efímeros (al ser apagado el programa todo lo que está en la RAM desaparece). La serialización nos permite guardar la información o estado actual del programa para que no se pierda al momento de apagar la computadora y a enviar información, podemos mandar un objeto a través de internet a otra computadora, pues la RAM no se envía por cable.

Manejo de archivos.

Para guardar esos bytes en el disco, en Java utilizamos flujo de datos (Streams). La forma más rápida y moderna de hacerlo es usando el paquete java.nio.file (NIO.2) que maneja mejor los errores del sistema operativo que el anterior java.io.File.

Cuando abrimos un archivo se consume un recurso del sistema operativo (llamado file handle). Si el programa lanza un error y el desarrollador olvida cerrar el flujo (con método close()), el archivo se queda “bloqueado”, si esto sucede ningún otro programa podrá leerlo o borrarlo y va a generar una fuga de memoria.

Lo solucionamos con (try-with-resources), al declarar la apertura del archivo entre los paréntesis del try, Java tiene que cerrarlo al terminar, sin importar si nuestra escritura fue exitosa o si el programa colapsó lanzando una excepción.

SerialVersionUID y transient.

SerialVersionUID actúa como control de versiones de la clase, si guardamos un objeto en disco y luego modificamos su código fuente (ej, agregar un nuevo campo), el ID guardado y el nuevo ya no van a coincidir. Al intentarlo de nuevo Java va a notar la diferencia y lanzará una InvalidClassException para corromper la memoria con datos incompatibles.

Cuando un campo es marcado como transient este le indica a la máquina virtual que inore esa variable por completo, es lo mejor para datos sensibles (contraseñas). Al serializar no se guarda en el disco y deserializar el objeto, el campo se inicia con su valor por defecto (null para objetos o 0 para números).

¿Dónde se ve esto en el código?

Serialización: En la ruta src/com/serializable/v2/Gorilla.java, implementamos Serializable en la clase qui se definió su serialVersionUID y se marca la comida favorita como un transient.

Persistencia: En los archivos PrincipalObjectOutput.java y PrincipalObjectInput.java se utiliza ObjectOutputStream y ObjectInputStream para guardar y leer la lista de gorilas desde el archivo data/gorillas.data.

Archivos y Streams (NIO.2): en la ruta de src/com/curso/streams/v1/PrincipalPath03.java se utiliza Files.newBufferedReader dentro de try-with-resources para copiar los datos de forma segura. En el archivo principalPath02.java se utiliza la API de Streams para mostrar la lectura perezosa o lazy.

¿Cómo correrlo?

En terminal desde la raíz del proyecto:

- 1- Compilar con “javaac -d . src/com/serializable/v2/*.java
- 2- Escribir datos: java com.serializable.v2.PrincipalObjectOutPut
- 3- Leer datos (y demostrar transient): java com.serializable.v2.PrincipalObjectInput